ASPAS E EXPANSÕES

O script de limpeza tinha uma linha só e rodou oito meses sem incidente — até o dia em que `DESTINO` *passou a valer* `/srv/Arquivos Antigos`*. Nenhum comando estava errado. Faltavam* ***duas aspas****.*

053
GUIAS LINUX

A LINHA QUE VOCÊ DIGITA NÃO É A LINHA QUE RODA

A LINHA DIGITADA	`cp ~/dados{1,2}.csv "$DEST"/$(date +%F)`
1 · CHAVES	`cp ~/dados1.csv ~/dados2.csv "$DEST"/$(date +%F)`
2 · TIL	`cp /home/kali/dados1.csv … "$DEST"/$(date +%F)`
3 · VARIÁVEIS, $(( )) E $( )	`cp /home/kali/dados1.csv … "/srv/Back up"/2026-08-03`
4 · DIVISÃO EM PALAVRAS	`[cp] [/home/kali/dados1.csv] [/home/kali/dados2.csv] [/srv/Back up/2026-08-03]`
5 · GLOBBING	`sem curinga nesta linha: nada muda`
6 · REMOÇÃO DAS ASPAS	as aspas somem — elas nunca chegam ao comando. O `cp` recebe 3 argumentos.

SEM ASPAS
`rm -rf $DESTINO/*`
`rm` `-rf` `/srv/Arquivos` `Antigos/*` → o 2º caminho é resolvido a partir do **diretório atual**

COM ASPAS
`rm -rf "$DESTINO"/*`
`rm` `-rf` `/srv/Arquivos Antigos/relatorio.pdf` → um argumento por arquivo real

Ensaie todo `rm -rf` com `echo` na frente. `echo rm -rf "$DESTINO"/*` mostra exatamente os argumentos que o comando receberia, sem apagar nada. O `/*` fica **fora** das aspas de propósito: é ele que precisa da etapa 5 para virar a lista de arquivos. **Aspas em volta do caminho, curinga fora** — é o que separa o *dado* (o valor da variável) da *sintaxe* (o curinga que você escreveu).

As etapas **4 e 5 são as perigosas**, e as duas acontecem **depois** de a variável virar valor. O shell não distingue o espaço que você digitou daquele que veio de dentro do valor — para ele é tudo a mesma linha. Aspas duplas não impedem a substituição da etapa 3; impedem que o resultado dela seja fatiado na 4 ou reinterpretado como curinga na 5.

AS FORMAS DE CITAR · E AS OUTRAS EXPANSÕES

FORMA	O QUE DESLIGA	O QUE SOBRA LIGADO
sem aspas	nada	tudo — inclusive a divisão e o curinga
`'simples'`	**tudo**	nada: cada byte é literal
`"duplas"`	divisão em palavras e *globbing*	`$var`, `$(cmd)`, `$((..))`, `\`
`\c`	só o próximo caractere	o resto da linha
`$'texto'`	como as simples, mas...	interpreta `\n`, `\t`, `\x41`

`$(( a + b ))`
Aritmética **só inteira**: `10/4` vale **2**. Decimal é trabalho de `bc`, `awk` ou `python3`.

`{1..5}` · `{a..e}` · `{0..20..5}`
Primeira etapa, logo **não enxerga variável**: `{1..$n}` sai literal — use `seq 1 "$n"`. Idioma clássico: `cp conf{,.bak}`.

`~` · `~root` · `~+`
Não expande dentro de aspas: `"~/dados"` é literal. Em script, `"$HOME/dados"`.

`<(comando)`
Entrega a saída como se fosse arquivo: `diff <(sort a) <(sort b)`. E `while … done < <(cmd)` é o laço **sem** subshell.

kali@kali: ~/lab-aspas

```
$ curinga="*"; echo $curinga      # lista TODOS os arquivos: a etapa 5 vê o VALOR
$ echo "$curinga"                 *
$ dono="$(stat -c %U "$(readlink -f /bin/sh)")"  # $( ) aninha; a crase `` nao
$ n=0; printf 'a\nb\nc\n' | while read -r _; do n=$((n+1)); done; echo $n
0  # o pipe roda o while num SUBSHELL — o contador voltou zerado
$ n=0; while read -r _; do n=$((n+1)); done < <(printf 'a\nb\nc\n'); echo $n
3  # com substituicao de processo, o laco fica no shell atual
$ find /tmp -name '*.log' -delete  # a lista NAO passa pelo shell: o jeito certo
```

O `eval` transforma dado em sintaxe — é a via curta para injeção de comando. Com `entrada="/tmp; rm -rf ~"`, o `eval "ls $entrada"` executa também o `rm`: o ponto e vírgula que estava no *dado* virou sintaxe. Defesas, em ordem: **1.** não use `eval` — quase sempre há array (`"${opcoes[@]}"`), `${!var}` ou `case` · **2.** valide por **lista de permitidos**, nunca removendo caracteres proibidos · **3.** passe como argumento e encerre as opções: `comando -- "$valor"` · **4.** `printf %q` quando montar linha for inevitável. O mesmo vale para `bash -c "$linha"`, `find -exec sh -c` e, sobretudo, `ssh host "cmd $VAR"` — que atravessa **dois** shells: o local expande, o remoto reinterpreta o resultado inteiro.

RESUMO RÁPIDO

- ✓ **Seis etapas fixas:** chaves → til → variáveis/aritmética/ `$( )` → divisão em palavras → *globbing* → remoção das aspas. Quase toda surpresa do shell é uma etapa fora do lugar que você imaginou.
- ✓ **Aspas duplas em toda expansão, sempre**, salvo motivo articulável. As simples tornam tudo literal; `$'…'` interpreta `\n` e `\t`.
- ✓ **`$(cmd)` no lugar da crase:** aninha, dispensa escapar barra e é inequívoco. Ele **corta as quebras de linha finais** e roda num subshell.
- ✓ **Nome de arquivo pode ter espaço, `;` e até quebra de linha.** Nunca analise a saída do `ls`: use *globbing* (`for f in *.txt`) ou `find -print0 | xargs -0`.
- ✓ `shellcheck script.sh` acha sozinho SC2086, SC2046, SC2006 e SC2045 — o hábito de maior retorno do volume.

NO KALI

Nome com espaço é rotina em perícia, não exceção. Evidência vinda de máquina Windows chega com `Área de Trabalho`, `Novo Documento (2).docx`, acentos e parênteses. Script de triagem sem aspas falha **parcialmente**: processa alguns arquivos, ignora outros e o relatório sai incompleto *sem erro visível*. Nesse contexto a combinação é obrigatória: `find -print0 | xargs -0` ou `find -exec … {} +` — o byte nulo é o único caractere que não pode aparecer num nome. **A saída de ferramenta do Kali não é contrato estável:** `nmap`, `nikto` e `gobuster` mudam formato entre versões e, num *rolling*, isso acontece sem aviso — um `grep`/`cut` na saída de tela quebra no próximo `full-upgrade`. Use saída estruturada (`nmap -oX`, `-oG`, `--json`) e `jq` em cima. E cuidado com quem *consome* as aspas: em `proxychains nmap -sT "$ALVO"` quem as come é o shell local, mas em `ssh host "cmd $ALVO"` um alvo com `;` no meio vira comando na máquina remota. Em teste autorizado é a diferença entre um script e um incidente — agir fora do escopo contratado é crime pela Lei 12.737/2012, ainda que por acidente de aspas.